



摘 要

在电子游戏领域，构建具备精确行为逻辑和出色决策能力且能与人类玩家进行高水平对抗的智能体，一直以来都是众多游戏制造商的核心关注点。随着绝大多数游戏规则的复杂化，仅依赖于特定逻辑和有限状态机策略的传统智能体已经难以达到玩家们的期望水平。本研究利用 Unity3D 游戏引擎及 Unity ML-Agents 工具包部署了一个本地强化学习环境，通过创建不同复杂度的三个对抗性游戏场景，检验智能体能否在实际游戏中达到开发者所期望的能力水平。实验采用了经典的 PPO 算法，在“中心化训练 + 去中心化决策”的架构下，系统探讨了多智能体如何利用强化学习的优势在对抗性游戏中进行实时和自动化的决策。本研究有效利用了电子游戏的虚拟特性，采用射线传感器作为数据输入来替代传统的参数和相机传感器，进而优化智能体的环境感知能力和决策精度。此外，本研究还引入了 ELO 评级系统代替团队奖励，以量化多个同属性和相同目标的智能体在对抗性游戏中的相对表现水平。

关键词：强化学习，多智能体系统，对抗性游戏，PPO 算法，ELO 评级系统



Abstract

Constructing intelligent agents with precise behavior logic and excellent decision-making abilities capable of high-level competition against human players has always been a core focus for many game developers. As the complexity of most game rules increases, traditional agents relying solely on specific logic and finite state machine strategies are increasingly failing to meet player expectations. This study leverages the Unity3D game engine and Unity ML-Agents toolkit to deploy a local reinforcement learning environment. By creating three adversarial game scenarios of varying complexity, we examine whether the agents can achieve the desired level of competence in actual gameplay. The experiment utilizes the classic PPO algorithm and explores how multiple agents can leverage the advantages of reinforcement learning to make real-time and automated decisions in adversarial games within a "centralized training + decentralized execution" framework. This study effectively utilizes the virtual nature of video games, employing ray sensors as data inputs to replace traditional parameter and camera sensors, thereby optimizing the agents' environmental perception and decision-making accuracy. Additionally, the study introduces the ELO rating system instead of team rewards to quantify the relative performance levels of multiple agents with the same attributes and goals in adversarial games.

Keywords: Reinforcement Learning, Multi-Agent System, Adversarial Game, PPO Algorithm, ELO Rating System



目 录

第一章 引言	1
1.1 研究背景	1
1.2 研究意义	1
第二章 相关技术	2
2.1 深度强化学习技术	2
2.2 单智能体强化学习算法	2
2.2.1 REINFORCE 算法	2
2.2.2 PPO 算法	4
2.2.3 Actor-Critic 算法	5
2.3 多智能体强化学习架构	6
2.3.1 “中心化训练” + “中心化决策”	6
2.3.2 “去中心化训练” + “去中心化决策”	7
2.3.3 “中心化训练” + “去中心化决策”	7
第三章 实验方法	8
3.1 实验平台	8
3.2 环境配置	8
3.3 状态传感器	9
3.3.1 参数传感器	9
3.3.2 相机传感器	9
3.3.3 射线传感器	10
3.3.4 比较与选择	11
3.4 自我博弈	12
3.4.1 ELO 评分系统	12
3.4.2 胜率估计	13
3.4.3 评分更新	13
3.5 课程学习	13
第四章 实验设计	14
4.1 智能体设计	14
4.1.1 动作空间设计	14
4.2 实验一：双人射击	14
4.2.1 状态空间设计	15



4.2.2	奖励函数设计	16
4.3	实验二：双人抓捕	16
4.3.1	状态空间设计	17
4.3.2	奖励函数设计	18
4.4	实验三：三人足球	18
4.4.1	状态空间设计	19
4.4.2	奖励函数设计	19
第五章	实验结果	20
5.1	实验参数设置	20
5.2	实验一：双人射击	20
5.3	实验二：双人抓捕	23
5.4	实验三：三人足球	25
第六章	结论与展望	27
6.1	主要结论	27
6.2	研究展望	29
参考文献		30



第一章 引言

1.1 研究背景

近年来，多智能体系统在许多领域中的实际应用越来越广泛，同时多智能体强化学习已经成为实现复杂智能体协作和竞争的重要手段。与单一智能体不同的点在于，多智能体强化学习领域里，智能体不仅需要学会观察周围环境，解释其他智能体的行为，还需要动态地调整策略，以适应环境的变化和其他智能体的行动。而对抗性的电子游戏恰恰为多智能体强化学习提供了一个理想的研究平台，这类环境充满了挑战性的动态特征，如不确定性、部分可观察性和智能体间高度的互动性。在这样的环境中，每个智能体既要与对手作斗争以争取利益，又要与其他智能体协作，共同实现更高的团队目标。近代的电子游戏如英雄联盟、炉石传说、星际争霸^[1]等，这些游戏不仅考验智能体的单一技能，如记忆力、策略规划和快速反应，还考验智能体的团队合作、竞争对抗、甚至是在充满变数的环境下进行长期规划和策略调整的能力。

1.2 研究意义

深入探讨对抗性游戏环境中的多智能体强化学习，具有重要的理论和实际意义。在理论领域，它丰富了我们对于智能体自主学习、决策制定过程以及智能体间信息通讯的理解。通过模拟复杂的对抗关系，研究人员不仅能够测试和改进现有的算法，还能够发现新的学习机制和智能行为模式。而从实际应用的角度来看，研究对抗性游戏环境中的多智能体强化学习能够直接在各类游戏中，为人类玩家提供不同层次水平的竞争对手或合作伙伴，从而丰富游戏体验并提供人类行为研究的数据。当然，多智能体强化学习在对抗性游戏中的应用，不仅限于提升电脑游戏的人工智能水平，更重要的是，这些研究成果可以迁移到现实世界的多种场景中，比如自动驾驶车辆的协调、无人机群的优化控制、复杂工业生产线的智能管理等^[2]。这些应用场景相比游戏环境更加复杂且具有不确定性，需要智能体间拥有更高层次的沟通、合作及自我适应能力。因此，对抗性游戏环境中多智能体强化学习的研究不仅能够推动理论的发展，更能够实现技术的跨界应用，为现实世界的各项任务提供合理的仿真模型，设计出更智能、适应性更强的人工智能系统。



第二章 相关技术

2.1 深度强化学习技术

强化学习的本质其实是让智能体在未知的环境中，通过反复尝试和不断试错来积累经验，最终做出对自己最有利的行为，我们可以使用马尔科夫决策模型的五元组 $\langle S, A, P, r, \gamma \rangle$ 来描述这个过程。其中 S 是智能体和环境共同形成的状态信息； A 是智能体所采取的动作集合； P 代表智能体的状态转移概率； r 是环境给予智能体的奖励； γ 则是未来奖励的折扣因子。而强化学习的目标，便是让智能体通过训练，找到一个最优策略 π^* ，使其每一步采取行为所获得的期望奖励值比其它任何策略 π 都要大。而目前的主流强化学习训练方式结合了深度学习在复杂高维数据处理上的强大能力^[3]：假设 π_θ 是智能体的目标策略，利用神经网络对其建模， θ 为模型的相关参数，输入为智能体所获得的状态 s ，输出为智能体所采取动作的 a 或者动作概率分布 $\pi(a|s)$ ，随后通过奖励反馈来不断优化 θ 以逼近目标策略。

2.2 单智能体强化学习算法

强化学习算法有基于价值和基于策略两种形式，前者每次选取价值最大的动作进行执行，训练出来的往往是确定性策略。但是考虑到游戏当中的最优策略通常是有随机性的，比如“石头-剪刀-布”游戏的最优策略是完全随机地选择三种手势中的任意一种，又或者在“扑克”游戏中，如果智能体总是在有好牌时加注，且在坏牌时弃牌，那么它的对手会很快识破其行为模式并加以利用。所以，在本章节中，主要介绍基于策略的强化学习算法：包括 REINFORCE 算法^[4]，PPO 算法^[5]，以及 Actor-Critic 算法^[6]。

2.2.1 REINFORCE 算法

考虑智能体在游戏当中的一次完整轨迹： $\tau = \{s_1, a_1, s_2, a_2, \dots, s_t, a_t\}$ ，该轨迹 τ 发生的概率为： $p_{\theta}(\tau)$ ，我们的目标是要使智能体在该游戏中所有轨迹的期望奖励 $\bar{R}_\theta$ 最大化，并采用梯度上升的方法 ($\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta$) 对策略参数 θ 进行更新：

$$\nabla \bar{R}_\theta = \sum_{\tau} R(\tau) \nabla p_{\theta}(\tau) \quad (2.1)$$

在实际计算过程中，我们显然无法模拟所有的轨迹来得到确切的期望，只能通过采样来获取近似解：

$$\begin{aligned}\nabla \bar{R}_\theta &= \sum_{\tau} R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau) \\ &= \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)] \\ &= \frac{1}{N} \sum_{n=1}^N R(\tau^n) \nabla \log p_\theta(\tau^n)\end{aligned}\quad (2.2)$$

注：上式利用了 $\nabla f(x) = f(x) \nabla \log f(x)$ 。

将 $\nabla \log p_\theta(\tau^n)$ 进一步化简，忽略只与环境相关的 $p(s_1)$ 以及 $p(s_{t+1}|s_t, a_t)$ ，只考虑和策略参数 θ 相关的 $p_\theta(a_t|s_t)$ ：

$$\begin{aligned}\nabla \log p_\theta(\tau) &= \nabla \left(\log p(s_1) + \sum_{t=1}^T \log p_\theta(a_t|s_t) + \sum_{t=1}^T \log p(s_{t+1}|s_t, a_t) \right) \\ &= \sum_{t=1}^T \nabla \log p_\theta(a_t|s_t)\end{aligned}\quad (2.3)$$

结合公式 (2.2) 和 (2.3) 可以得到 $\nabla \bar{R}_\theta$ 的计算公式：

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n) \quad (2.4)$$

直观地理解上式，假设智能体在完成轨迹 τ 后的总奖励 $R(\tau) > 0$ ，则调整策略 θ 以增加该轨迹中所有状态动作对的概率 $p_\theta(a_t|s_t)$ ；反之，如果 $R(\tau) < 0$ ，则减少智能体在 s_t 执行 a_t 的可能性。

在实际应用当中，会将公式 (2.4) 改写为：

$$\begin{aligned}\nabla \bar{R}_\theta &\approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \left(\sum_{t'=t}^{T_n} \gamma^{t'-t} r_{t'}^n - b \right) \nabla \log p_\theta(a_t^n | s_t^n) \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} A^\theta(s_t^n, a_t^n) \nabla \log p_\theta(a_t^n | s_t^n) \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} [A^\theta(s_t, a_t) \nabla \log p_\theta(a_t | s_t)]\end{aligned}\quad (2.5)$$

我们将 $A^\theta(s_t, a_t)$ 这项称为优势函数，用它来替换 $R(\tau)$ ，一方面是因为在很多游戏中 $R(\tau) \geq 0$ ，无论如何都会使采样得到的 (s, a) 概率提升，而通过减去基线 b ($b \approx E[R(\tau)]$) 的方法，我们就可以在训练的过程中让权重项变得有正有负，从而减小低收益选择的概率。

另一方面是因为在一次完整的轨迹当中，即使最终的结果是好的，也并不代表其中的每个选择都是好的，我们可以将原先整场游戏的奖励和，改成从 t 时刻到游戏结束的折扣奖励和，来计算该 (s_t, a_t) 执行以后对整体的贡献，用于判断其选择的好坏。然而，该算法更容易让智能体得到局部最优而非全局最优，因为每次采样的差异性会导致计算出的梯度存在很大方差，从而影响更新稳定性^[7]。

2.2.2 PPO 算法

公式 (2.2) 中的 $\mathbb{E}_{\tau \sim p_\theta(\tau)}$ 是对策略 π_θ 采样的轨迹 τ 求期望，如果执行了一次梯度更新操作， θ 的改变会导致当前的 $p_\theta(\tau)$ 无法再次使用，这使得策略梯度算法在每次更新参数后都需要重新与环境交互以获取新轨迹，从而花费大量时间在数据采样上。如果我们假设 θ 有能力去学习另外一个策略 θ' 所采样的数据，那么只需要让 θ' 进行多次轨迹的采样，然后利用这些数据来更新 θ ，便可以节省较多的采样时间。

下式使用了重要性采样对公式 (2.5) 进行修正，同时假设了 $A^\theta(s_t, a_t) = A^{\theta'}(s_t, a_t)$ 以及 $p_\theta(s_t) = p_{\theta'}(s_t)$ ，前者是因为得到的优势函数是根据 θ' 采样所估计出来的，后者是因为在绝大多数游戏当中智能体获得的状态与采取的动作之间是没有太大关系的。

$$\begin{aligned}\nabla \bar{R}_\theta &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(s_t, a_t)}{p_{\theta'}(s_t, a_t)} A^\theta(s_t, a_t) \nabla \log p_\theta(a_t | s_t) \right] \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)} \frac{p_\theta(s_t)}{p_{\theta'}(s_t)} A^{\theta'}(s_t, a_t) \nabla \log p_\theta(a_t | s_t) \right] \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \nabla \log p_\theta(a_t | s_t) \right]\end{aligned}\quad (2.6)$$

在我们对 θ 求梯度时， $A^{\theta'}(s_t, a_t)$ 和 $p_{\theta'}(s_t)$ 可以视为常数，同时结合 $\nabla f(x) = f(x) \nabla \log f(x)$ 反推原来的目标函数可得：

$$J^{\theta'}(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right] \quad (2.7)$$

重要性采样的前提是 $p_\theta(a_t | s_t)$ 和 $p_{\theta'}(a_t | s_t)$ 不能相差太多，我们可以通过目标函数中进行范围限制，以保证策略参数的更新差距不会太大：

$$\begin{aligned}J_{\text{PPO}}^{\theta'}(\theta) &\approx \sum_{(s_t, a_t)} \min \left(\frac{p_\theta(a_t | s_t)}{p_{\theta^k}(a_t | s_t)} A^{\theta'}(s_t, a_t), \right. \\ &\quad \left. \text{clip} \left(\frac{p_\theta(a_t | s_t)}{p_{\theta'}(a_t | s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) A^{\theta'}(s_t, a_t) \right)\end{aligned}\quad (2.8)$$

如果 $A^{\theta'}(s_t, a_t) > 0$, 那么说明 (s_t, a_t) 的价值相对较高, 优化这个目标函数会使 $\frac{p_{\theta'}(a_t|s_t)}{p_{\theta}(a_t|s_t)}$ 不断增大, 但不会超过 $1 + \varepsilon$; 反之, 如果 $A^{\theta'}(s_t, a_t) < 0$, 优化这个式子会使 $\frac{p_{\theta'}(a_t|s_t)}{p_{\theta}(a_t|s_t)}$ 不断减小, 但不会低于 $1 - \varepsilon$ 。

2.2.3 Actor-Critic 算法

考虑公式 (2.5) 中的优势函数 $A^{\theta}(s_t, a_t)$, 其表示的是智能体在 t 时刻采取行动后, 直到游戏结束所获得折扣奖励和, 这实际上就是动作价值函数 $Q(s_t^n, a_t^n)$ 的定义; 而其中的基线 b 则可以用状态价值函数 $V(s_t^n)$ 来表示:

$$\begin{aligned}\nabla \bar{R}_{\theta} &\approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} (Q_{\pi}(s_t^n, a_t^n) - V_{\pi}(s_t^n)) \nabla \log p_{\theta}(a_t^n | s_t^n) \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} (r_t^n + \gamma V_{\pi}(s_{t+1}^n) - V_{\pi}(s_t^n)) \nabla \log p_{\theta}(a_t^n | s_t^n)\end{aligned}\quad (2.9)$$

至此, 只要我们在策略网络基础上, 再额外引入价值网络用来估计 $V_{\pi}(s)$, 那么我们就可以做到每一步都能更新网络, 而不用像 REINFORCE 算法或者 PPO 算法那样需要经过一次完整的采样后才能更新。



图 2.1 Actor (策略网络) 和 Critic (价值网络) 的关系^[8]

策略网络的参数 θ 按照公式 (2.9) 来更新, 价值网络的参数 ω 则利用时序差分的学习方式, 以均方误差损失函数来更新:

$$\nabla_{\omega} \mathcal{L}(\omega) = -(r + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)) \nabla_{\omega} V_{\pi}(s_t) \quad (2.10)$$

2.3 多智能体强化学习架构

之前介绍的算法都仅适用于单智能体的情形，而在绝大部分游戏中，都会有多个智能体进行交互，因此任务也会变成多智能体强化学习。接下来将介绍三种多智能体强化学习的训练架构。

2.3.1 “中心化训练” + “中心化决策”

最直接的训练方式就是将多个智能体视作一个超级智能体，由中央控制器来收集 t 时刻所有智能体的状态，利用中央价值网络做评估，然后通过中央控制的策略网络给到各个智能体应该做的动作，并以此不断更新^[9]。而在训练完成后的决策阶段，智能体不具备独立的决策能力，只能通过将自身的观察传输给中央控制器，在中央控制器得到了所有智能体的观测结果后传回才能行动。

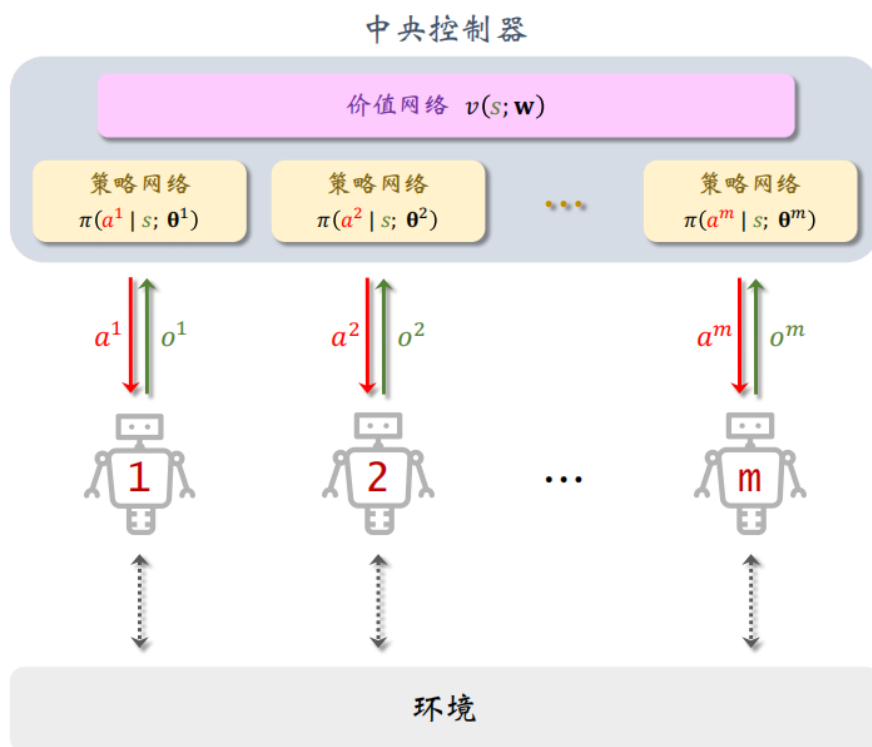


图 2.2 “中心化训练 + 中心化决策” 架构^[10]

这种训练方式可以完全照搬单智能体的训练算法，在一定程度上能保证算法的收敛性，但是需要考虑所有智能体的状态合集是否会产生维度爆炸的后果，以及决策阶段智能体和中央控制器要以何种方式做通讯。

2.3.2 “去中心化训练” + “去中心化决策”

与完全中心化相反的训练方式便是让每个智能体拥有自己的策略网络和价值网络，然后进行单独的训练。且在训练完成之后，智能体不再需要原先的价值网络做评估，可以直接利用自身的策略网络进行决策。

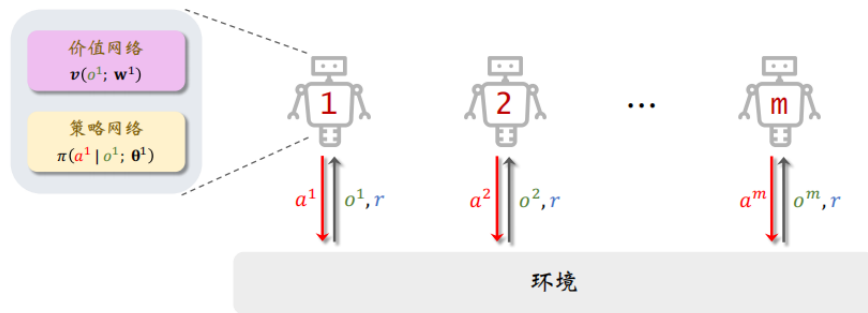


图 2.3 “去中心化训练 + 去中心化决策” 架构^[10]

这样的做法无需考虑通信问题，有很快的训练速度，但是忽视了多智能体之间的关联，使得训练环境变成非稳态，无法保证训练的收敛性。

2.3.3 “中心化训练” + “去中心化决策”

目前最为流行的多智能体架构实则是“中心化训练” + “去中心化决策”的组合。“中心化训练”使得智能体能够学习到更优秀的策略^[11]，而“去中心化决策”避免了智能体与中央控制器之间的通信问题，可以做到实时决策。

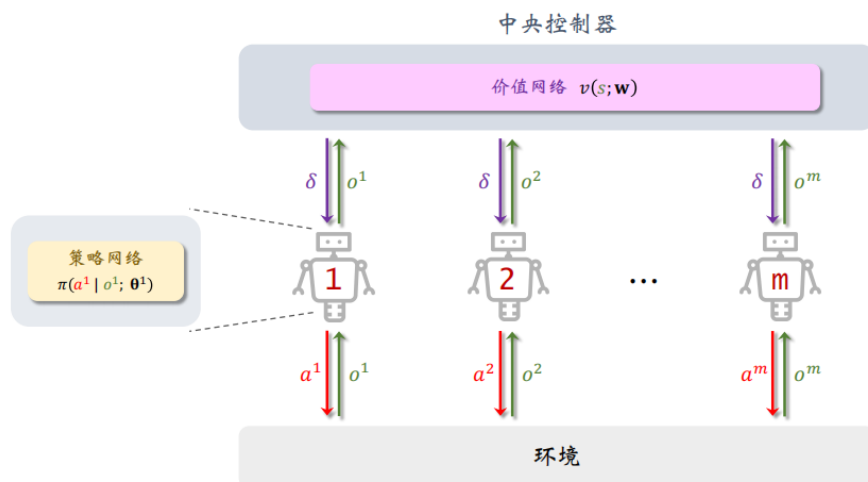


图 2.4 “中心化训练 + 去中心化决策” 架构^[10]

第三章 实验方法

3.1 实验平台

本实验基于 Unity 游戏引擎，利用 Unity Machine Learning Agents Toolkit（简称 Unity-ML）在 3D 空间中进行智能体的强化学习训练。Unity-ML 是 Unity 官方的一个开源项目，它使得用户能够自行创建环境，并通过 Python API 实现特定的算法来训练智能体^[12]。

3.2 环境配置

本实验在实验室主机电脑上完成，相关配置如表 (3.1) 所示：

表 3.1 硬件参数

硬件设备	型号
CPU	12th Intel(R) i9-12900K(24 CPUs)
GPU	NVIDIA GeForce RTX 4090
内存	4×32GB DDR5 4800MHz
硬盘	9TB

实验相关软件信息如表 (3.2) 所示：

表 3.2 软件参数

软件环境	版本
操作系统	Ubuntu 22.04.4 LTS
Python	3.9
C#	8.0.100
CUDA	11.8
Pytorch	2.3.0
Tensorboard	2.16.2
Unity	2022.3.23f1
Unity-ML	3.0.0



3.3 状态传感器

游戏本质上是一个连续的过程，这就意味着在每个瞬间，都需要将当前状态的信息传递给智能体。同时，为了确保智能体能够有效地完成其任务，它观察到的信息必须包含执行任务所需的所有必要细节。基于此，在 Unity 中实现了三种不同类型的状态传感器，以适应各类游戏场景需求^[13]。

3.3.1 参数传感器

最基础的状态传递方式是将所需要的参数整合后以向量的形式告诉智能体，这些参数往往是一些常见的基础数据类型，如布尔型、整型、浮点型等。需要注意的点在于智能体所得到的观测值必须包含相同数量的元素，并且必须始终以相同的顺序排序，如果环境中观察到的实体数量可以变化，则可以在调用中为特定观察中的任何缺失实体填充零，或者可以将智能体的观察限制为固定子集。

3.3.2 相机传感器

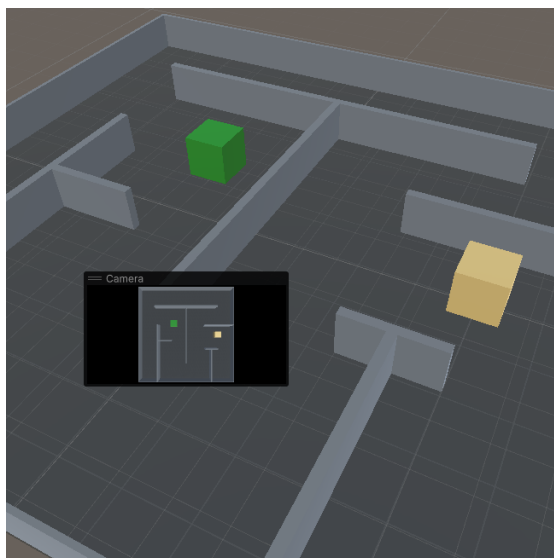


图 3.1 相机传感器俯视角示例

相机传感器是通过捕捉游戏世界的视觉信息来提供给智能体状态数据的一种方式，这种方法模拟了人类或动物通过视觉接收环境信息的方式，可以直接使用 Unity 中的相机来捕获当前视图的图像，并将其作为一个二维数组传递给智能体。相机传感器需要设置的相关参数如表 (3.3) 所示：



表 3.3 相机传感器参数

参数名	释义
Width	摄像机视图的宽度。
Height	摄像机视图的长度。
Grayscale	摄像机视图是否为灰度图。
Stacks	堆叠先前观察结果的数量。

相机传感器所生成的状态空间数量为：

$$\text{Stacks} \times \text{Width} \times \text{Height} \times \text{Channels} \quad (3.1)$$

3.3.3 射线传感器

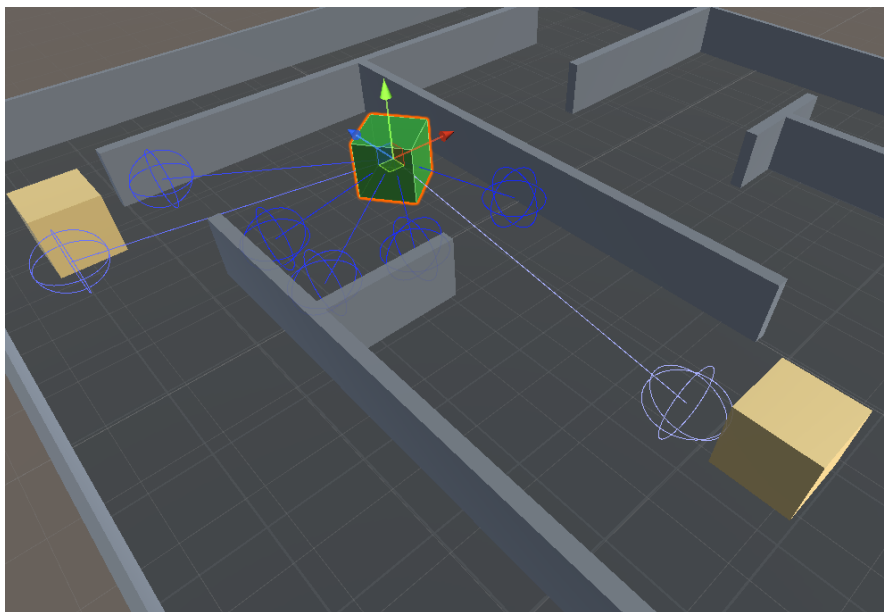


图 3.2 射线传感器俯视角示例

射线传感器是智能体向外发射虚拟的“射线”与环境中的对象进行相交检测来获取信息，这使得智能体能够对附近有一个大致的空间感知，了解周围敌人或障碍物的位置。射线传感器需要设置的相关参数释义如表 (3.4) 所示，其所生成的状态空间数量为：

$$\text{Stacks} \times \text{Rays Per Direction} \times \text{Num Detectable Tags} \quad (3.2)$$



表 3.4 射线传感器参数

参数名	释义
Detectable Tags	射线能检测到的物体标签集合。
Ray Length	智能体最远能检测到的物体距离。
Max Ray Degrees	正前方向左右覆盖的射线最大角度范围。
Rays Per Direction	每个角度范围内的射线条数。
Sphere Cast Radius	随射线投射出去的碰撞球体的半径。
Ray Layer Mask	射线能检测到的层级类型。
Start Vertical Offset	射线初始点垂直偏移距离
End Vertical Offset	射线结束点垂直偏移距离
Stacks	堆叠先前观察结果的数量。

3.3.4 比较与选择

选择适当的状态传感器对于开发出能有效在游戏环境中运作的智能体极为关键，同时也应该兼顾训练的时长与资源消耗。参数传感器主要用于捕捉环境中的静态或基本信息，例如位置、速度或其他直接量化的数值。这种类型的数据虽然对于某些决策过程很有用，但它们提供的信息较为局限，不能全面反映复杂环境或与其他对象间的动态交互情况。相机传感器提供的视觉图像数据虽然与人类的观察方式最为接近，但处理和解析这种数据需要大量的计算资源，仅考虑一个长宽均为 32 像素的彩色三通道摄像机，根据公式 (3.1) 计算其 1 帧的状态空间大小即为 $32 \times 32 \times 3 = 3072$ ，无论是训练还是实际运行都需要较高的硬件配置。射线传感器可以检测智能体视线范围内的障碍物、地形以及其它对象，所获得的信息也更为动态和丰富，可以帮助游戏智能体更好地理解 and 预测环境中可能发生的变化。另外，射线传感器既没有像相机那样有太高维度的输入，行为模式相比单纯的数据参数也更容易学习。

因此，对于游戏开发者而言，射线传感器结合了效率、动态环境感知能力以及实时信息处理的优势，其综合表现在多数游戏训练场景中都是最合适的选择，尤其在资源有限且对实时交互性要求较高的对抗性游戏环境中。



3.4 自我博弈

在对抗性游戏中，如果对立的双方具有同样的属性和目标，那么累积环境奖励可能并不是跟踪智能体学习进度的有效指标，因为对局的情况在很大程度上取决于对手水平的强弱。

而自我博弈，是指让智能体通过与自己的历史版本进行对战，以此不断学习和提高的方法^[14]。这种方法的核心优势在于绝大多数对抗性游戏都具有无模型且缺少专家数据的特点，智能体需要通过游戏过程积累经验，不断学习新的策略用于见招拆招，乃至探索出人类未知的游戏技巧。然而，在自我博弈的过程中，可能会出现某个智能体策略的暂时领先他人的情况，这会导致其他人学习困难，甚至学习不到有效的策略，从而不收敛。针对这一问题，一种有效的解决方法是在每轮开始时，随机选择一个历史版本的自己进行比赛，这样做能使学习过程更为稳定，策略更具有鲁棒性，还能有效缓解因实力不平衡导致的学习停滞现象。

3.4.1 ELO 评分系统

ELO 评分系统基于这样一个假设：游戏的胜负可以预测玩家之间的实力差异^[15]。玩家根据比赛结果获得或失去分数，胜过预期水平更高的对手能得到更多分数，而输给预期水平更低对手则会失去更多分数。

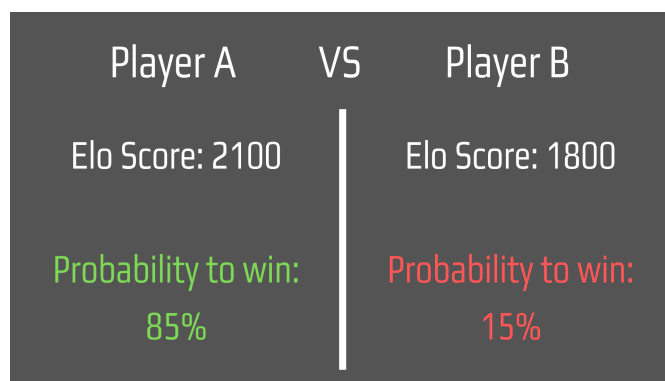


图 3.3 玩家 A/B 的 ELO 分数为 2100/1800，则他们的对抗胜率为 85%/15%

所有玩家初始的 ELO 分数均为 1200 分，在对抗性游戏中，ELO 分数取决于智能体相对于所面对的特定对手群体的表现。通常，如果智能体的 ELO 分数稳定增长，最终达到 1400 分左右，就可以视为它正在良好地学习和进步。



3.4.2 胜率估计

假设有玩家 A 的 ELO 分数是 R_A ，他对手玩家 B 的 ELO 分数分别是 R_B ，则玩家 A 的胜率估计为：

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \quad (3.3)$$

3.4.3 评分更新

玩家 A 在进行过一轮对抗后，他的 ELO 分数会更新为 R'_A ：

$$R'_A = R_A + K(S_A - E_A) \quad (3.4)$$

其中 K 为调整系数， S_A 为玩家 A 的实际表现分，如果获胜 S_A 为 1，失败则 S_A 为 0。假设调整系数 K 为 20，1800 分的玩家如果战胜了 2100 分的玩家，则他的 ELO 分会更新为 $1800 + 20 \times (1 - 0.15) = 1817$ 。

3.5 课程学习

在绝大部分游戏中，奖励只会在成功通关后给出。例如，在一个复杂的平台跳跃游戏中，智能体的目标是从起点开始，通过跳跃和避开障碍到达终点。如果仅在到达终点时给出奖励，那么在早期探索阶段智能体可能会因为不断失败而学习不到有价值的策略。为了应对这个问题，可以在训练的初始阶段先为智能体提供探索性的奖励^[16]，比如智能体如果成功跳跃，可以给予它少量奖励，再进一步地，当它避开一个障碍或达到关卡的中间点，再给予一部分奖励。这样的密集的奖励能够鼓励智能体执行特定的有益行为，如跳跃和避开障碍，从而帮助它学习达到最终目标所必需的基本动作。随着训练进程的推进，这些额外的探索奖励应逐渐减少直至消失，最终只保留完成整个关卡时的奖励。这种像课程一样逐渐递进的训练策略，能够确保智能体在最初学习到基本的游戏操作，然后逐步专注于优化通关策略，提高其在游戏中的整体表现。



第四章 实验设计

4.1 智能体设计

实验所使用的代表玩家的智能体三维模型如下图 (4.1) 所示，双方的外观结构均相同，主要用蓝色与紫色进行队伍区分。

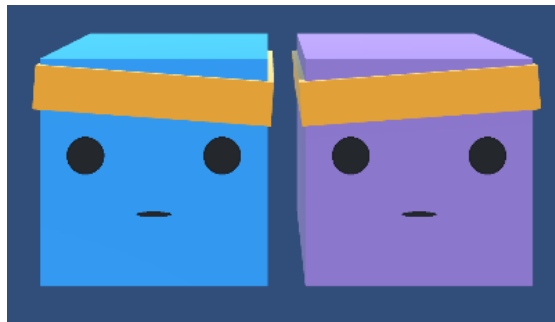


图 4.1 智能体三维模型图

4.1.1 动作空间设计

智能体的基础动作空间均采用了 3 组离散动作，根据观测结果每帧连续控制，不同组之间可以独立采取行动，且互不影响：

表 4.1 智能体离散动作空间组

参数名	释义
moveX	0 表示不移动，1 代表朝 x 轴正方向，2 代表朝 x 轴负方向
moveZ	0 表示不移动，1 代表朝 z 轴正方向，2 代表朝 z 轴负方向
rotateY	0 表示不旋转，1 代表绕 y 轴正方向，2 代表绕 y 轴负方向

4.2 实验一：双人射击

这是一场对称性的 2v2 射击比赛，玩家只能在己方的半区行动，目标是发射子弹击中对方玩家，同时避免对方发射子弹击中己方玩家。如下图所示，在双人射击的场景中，除了基础的建筑墙和地板之外，还增加了可供双方玩家躲避子弹的挡板，以及区分双方场地的透明隔离板。

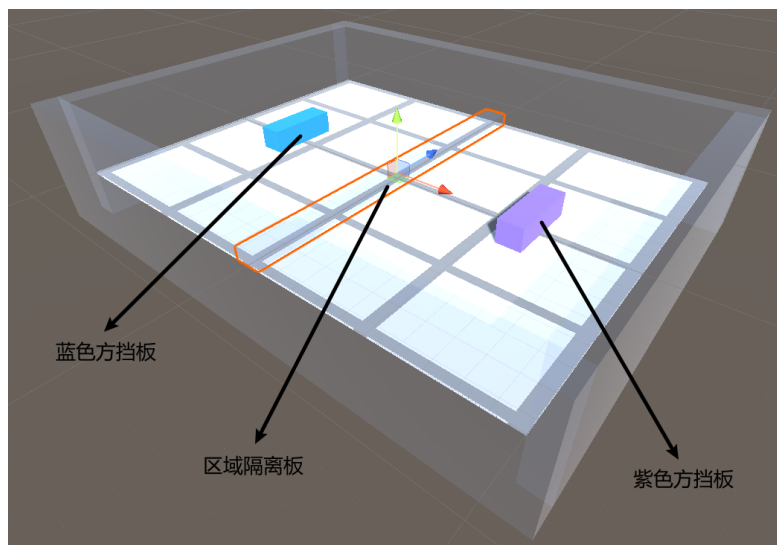


图 4.2 双人射击：三维场景效果图

4.2.1 状态空间设计

在射线传感器方面，让智能体向前方 200 度的范围内发射了 11 条射线，且向后方 90 度的范围内发射了 3 条射线，每条射线带有 0.5 半径的球型体积。这样的设计考虑了智能体在游戏过程中不仅需要感知前方的敌人位置，还需考虑侧方与后方的队友位置；前方 200 度范围内的广泛射线覆盖以及后方 90 度范围内较少的射线数量，衡量了进攻和防守的决策优先级以及对计算资源的节约考量。智能体的每条射线能够检测的标签如表 (4.2) 所示，同时保留连续 3 帧的信息作为输入。根据公式 (3.2) 计算出智能体前方射线的状态空间大小为 264，后方射线的状态空间大小为 72，总共为 336。

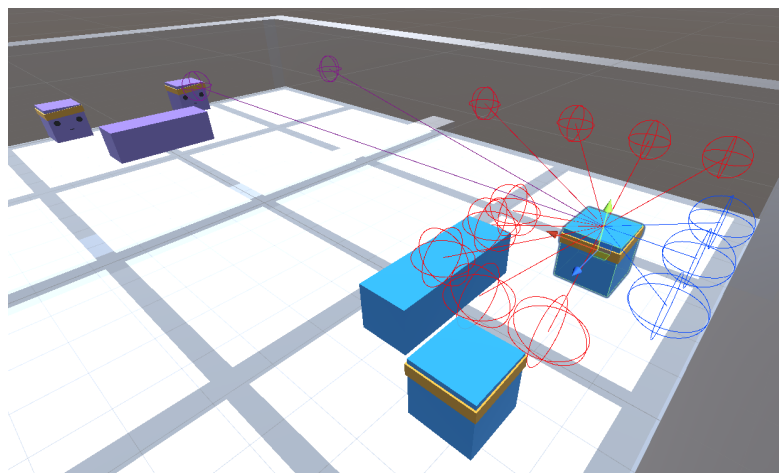


图 4.3 双人射击：射线传感器示例



表 4.2 双人射击：检测标签

参数名	释义
ball	子弹
wall	四周墙体
shelter	挡板
blueAgent	蓝方智能体
purpleAgent	紫方智能体
invisibleWall	区域隔离板

4.2.2 奖励函数设计

在该实验中，智能体的目标比较简单，故奖励函数也设置得较为简单。如果击中了对方，则在团队奖励 +1 的同时，扣除时间惩罚项 $\frac{Current\ Step}{Max\ Step}$ ；如果被对方击中，则团队奖励 -1。任意一方的一名队员被击中，则游戏结束。

4.3 实验二：双人抓捕

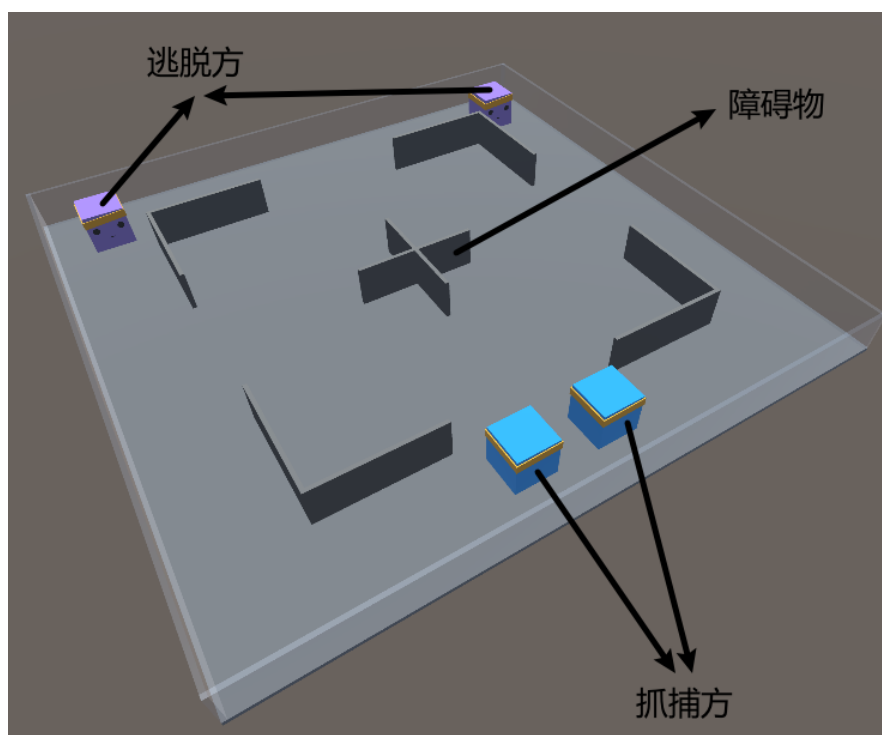


图 4.4 双人抓捕：三维场景效果图

这是一场对称性的 2v2 抓捕比赛^[17]，追捕方要在特定时间内抓到逃脱方，而逃脱方要尽可能避免被追捕方抓到。如上图 (4.4) 所示，在双人抓捕的场景中，除了基础的建筑墙和地板之外，还增加了许多障碍物供双方进行博弈。

4.3.1 状态空间设计

在射线传感器方面，让智能体向 240 度的范围内发射了 19 条射线，每条射线带有 0.25 半径的球型体积。这样的设计主要考虑了智能体在游戏过程中的第一视角以前方感知为主导，侧后方感知占比较小，同时也能给到抓捕方视线死角，给逃脱方更多灵活决策的可能性。智能体的每条射线能够检测的标签如表 (4.3) 所示，并保留连续 3 帧的信息。根据公式 (3.2) 计算出智能体的状态空间大小为 $3 \times 19 \times 5 = 285$ 。

表 4.3 双人抓捕：检测标签

参数名	释义
wall	四周墙体和障碍物
catcher	抓捕方智能体（蓝方）
runner	逃脱方智能体（紫方）

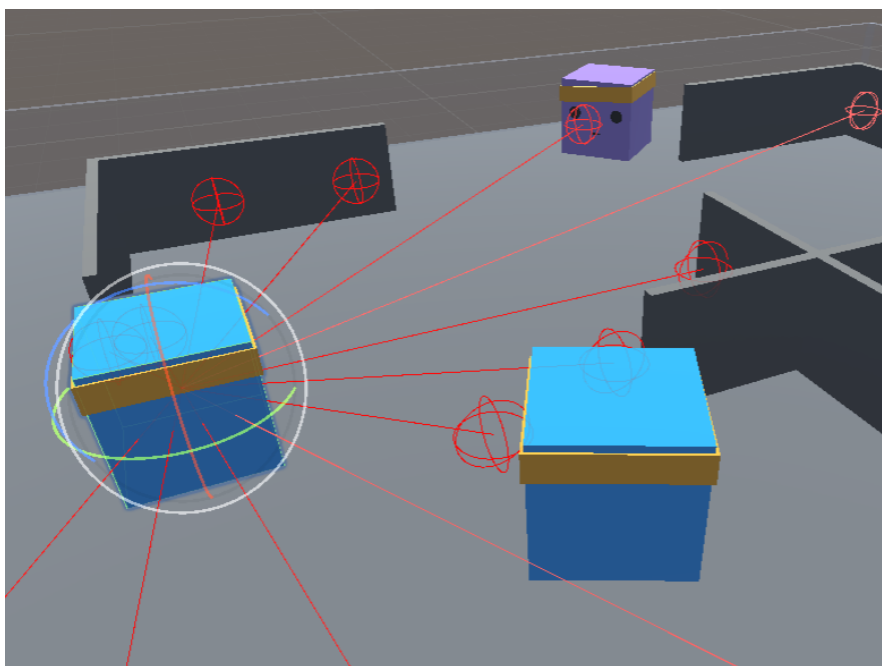


图 4.5 双人抓捕：射线传感器示例

4.3.2 奖励函数设计

对于抓捕方来说，每行动一步个人奖励 $-\frac{1}{Max\ Step}$ ，抓到逃脱方的智能体个人奖励 +1.5；对于逃脱方来说，每行动一步个人奖励 $+\frac{1}{Max\ Step}$ ，被抓捕方抓到的智能体个人奖励 -1。如果游戏到达最大步长或结束，则存活的逃脱方数量标记为 n ，抓捕方团队奖励为 $0.75n - 0.5$ ，逃脱方团队奖励为 $1 - 0.75n$ 。抓捕方个人奖励略大于逃脱方个人奖励是因为在训练刚开始的探索阶段，抓捕方很难获得奖励，而逃脱方即使原地不动也能依靠拖时间获取到 +1 的满额奖励。

表 4.4 双人抓捕：团队奖励

结果	抓捕方团队	逃脱方团队
抓到 0 人（逃脱 2 人）	-0.50	+1.00
抓到 1 人（逃脱 1 人）	+0.25	+0.25
抓到 2 人（逃脱 0 人）	+1.00	-0.50

4.4 实验三：三人足球

这是一场对称性的 3v3 足球比赛，双方队伍在标准的足球场内对战，每队初始配有 2 名离中场较近的队员和 1 名离球门较近的队员。为了简化游戏难度，智能体只有在向前移动时拥有踢球的能力，其它方向的触球不会产生力。

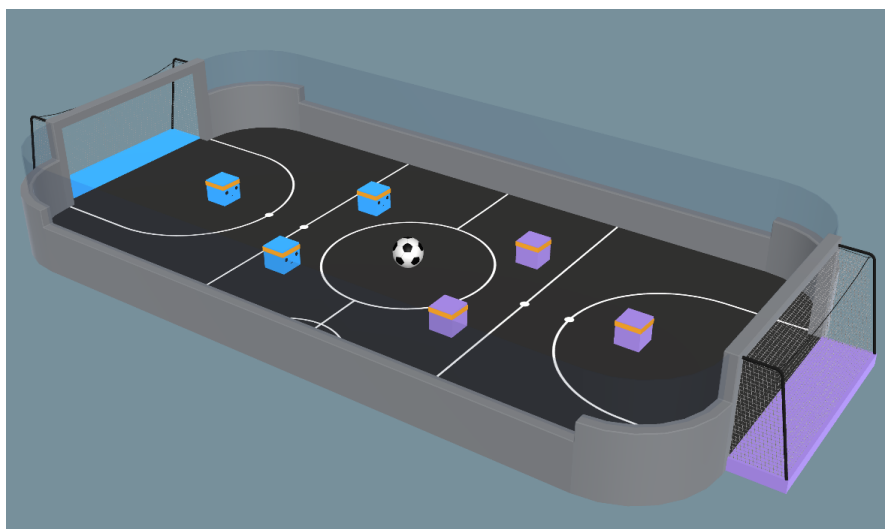


图 4.6 三人足球：三维场景效果图



4.4.1 状态空间设计

在射线传感器方面，考虑到单一射线传感器会存在阻挡问题（比如：队友身后有球，但智能体只探测到了队友，但没有探测到球），故采用两组射线传感器结合的方法。第一个射线传感器只探测最关键的足球，第二个射线传感器则探测队友、对手和墙壁等其它信息。

表 4.5 三人足球：射线传感器（一）检测标签

参数名	释义
ball	足球

表 4.6 三人足球：射线传感器（二）检测标签

参数名	释义
wall	墙壁
blueGoal	蓝色方球门
purpleGoal	紫色方球门
blueAgent	蓝色方球员
purpleAgent	紫色方球员

其中，第一个射线传感器向 360 度范围发射了 13 条射线，而第二个射线传感器向前方 120 度的范围内发射了 11 条射线，向后方 90 度的范围内发射了 3 条射线，每条射线带有 0.5 半径的球型体积。这样的设计主要考虑了足球本身在游戏中的重要性，以及足球队员各方位的感知能力差异，同时也给到了一定的视线死角，为智能体创造灵活决策的可能性。根据公式 (3.2) 计算出总的状态空间大小为 $3 \times 13 \times 3 + 3 \times 14 \times 7 = 411$ 。

4.4.2 奖励函数设计

除了最基础的进球奖励 +1，丢球奖励 -1 之外，增加了足球朝敌方球门移动增加奖励，朝己方球门移动减少奖励的做法，来避免奖励过于稀疏的情况。同时，引入课程学习的技巧，在训练初期智能体只要在向前移动时碰到球便可以获得 +0.1 的奖励，以让其初步掌握踢球的方法。



第五章 实验结果

5.1 实验参数设置

实验使用的深度强化学习算法为 PPO，优势函数选用 $r_t + \gamma V_{s_{t+1}} - V_s$ ，故需要同时训练策略网络和价值网络。整体架构为“中心化训练”+“去中心化决策”，设定的共同超参数如下表所示：

表 5.1 超参数列表

超参数	释义	值
max_steps	训练步数	10000000
learning_rate	PPO 算法中策略网络和价值网络的学习率	0.0003
epsilon	PPO 算法的裁剪比例，控制新旧策略间的最大偏差	0.2
gamma	未来奖励的折扣率	0.99
batch_size	每次梯度更新所用的样本数量	512
buffer_size	存储在经验回放池中的样本容量	5120
hidden_units	神经网络的隐藏层节点数量	256
num_layers	神经网络的隐藏层层数	3

5.2 实验一：双人射击

在双人射击实验中，由于双方智能体的行为模式相同，因此可以直接根据 ELO 评分来衡量智能体的决策水平。如下图所示，智能体的 ELO 分数在实验过程中稳步增长，在三百万步时达到 1300 并逐渐收敛维持在了该水平区间。

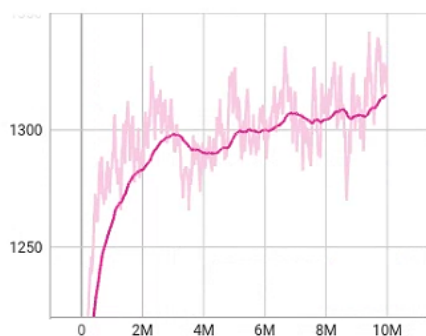


图 5.1 双人射击：ELO 评分



在训练完成后，将模型导入 Unity 中对智能体的实际行为进行了结果检验，并结合其实际行为绘制出了如下的策略仿真图。

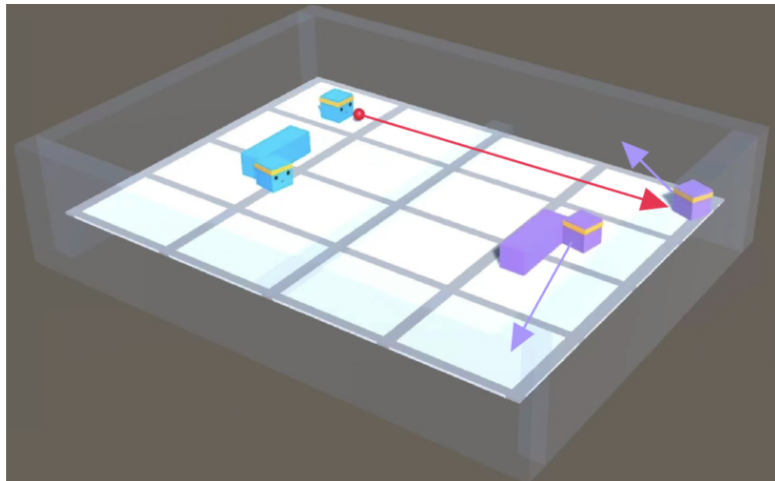


图 5.2 双人射击：策略仿真（一）

在蓝色方智能体瞄准紫色方智能体并发射子弹后，紫色方的两个智能体都会在第一时间向侧方移动以规避子弹。这说明智能体在训练过程中已经初步具备了瞄准能力以及躲避能力。

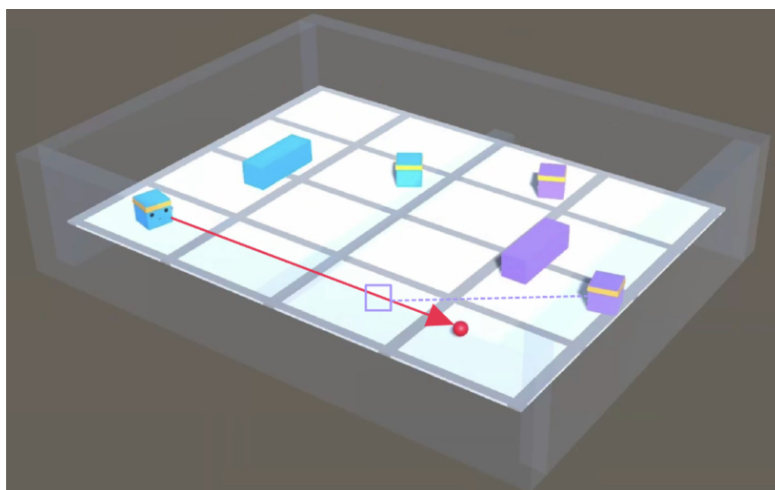


图 5.3 双人射击：策略仿真（二）

在发现对方朝自身发射子弹后，智能体也会试图躲避到障碍块的后方。这说明智能体在训练的过程中也懂得了利用环境来为自己创造优势。

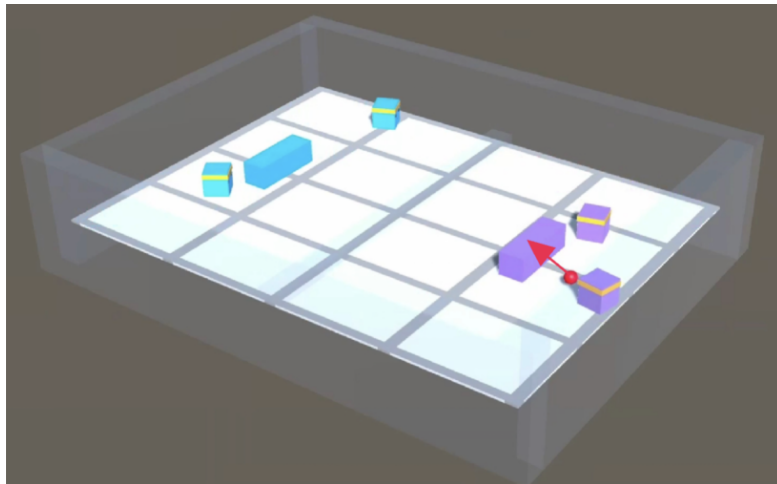


图 5.4 双人射击：策略仿真（三）

然而，由于在训练时并没有设计和障碍物相关的奖励函数，因此智能体虽然理解了环境中的障碍物会对自己产生帮助，但仍然会出现躲在障碍物后面，同时又向障碍物发射子弹的场景。

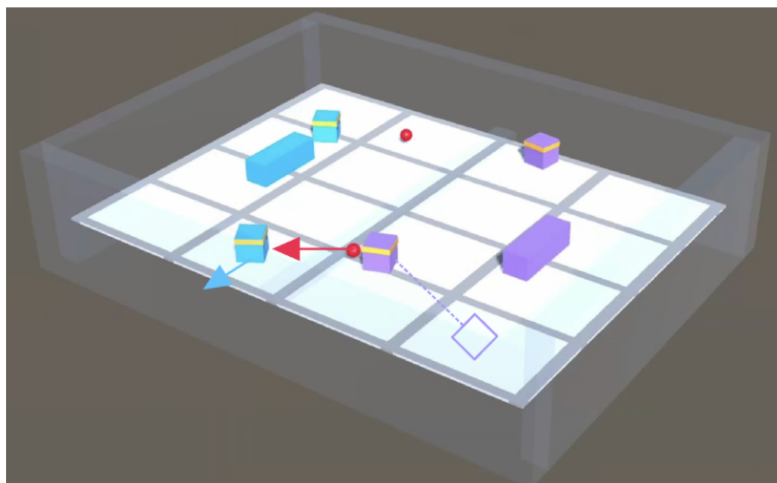


图 5.5 双人射击：策略仿真（四）

在部分对局中，智能体会选择冲到前场再发射子弹，利用双方距离变短的特性使得对手在第一时间来不及躲避子弹。智能体的这一策略表现是在设计之初并没有预料到的，也是绝大多数人类玩家在进行游戏的过程中可能并不会采取的一种策略。这说明了在对抗性游戏中利用强化学习，有可能训练出非预设的智能体策略，从而提供更加新颖和多样的玩法。



5.3 实验二：双人抓捕

在双人抓捕实验中，由于抓捕方和逃脱方两个团队属性不同，无法用 ELO 评分来衡量两方的水平，故通过团队奖励的方式来检验智能体的决策水平。其中蓝色线条为抓捕方，紫色线条为逃脱方。由于场景设置和初始奖励函数的设定，抓捕方在没有经验积累的情况下，其难度是远远高于逃脱方的，也因此抓捕方的团队奖励在训练开始时总是负的，而逃脱方总是正的。随着训练的不断进行，可以很明显地看到双方的团队奖励在不断升高。由于双方在同一个对局中采样，在三百百万步时，抓捕方学习到了有效的抓人策略，因此团队奖励开始变正，而逃脱方团队奖励开始降低，直到接近一千万步时收敛。

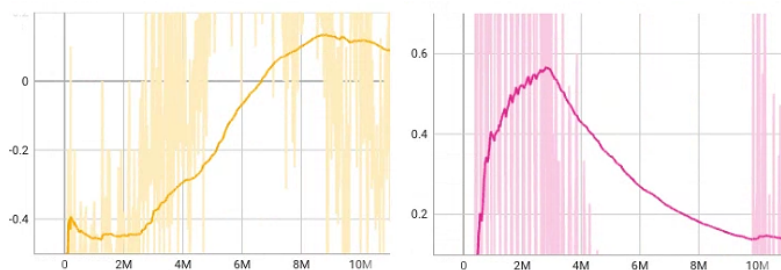


图 5.6 双人抓捕：左侧为抓捕方团队奖励，右侧为逃脱方团队奖励

策略仿真图如下所示：

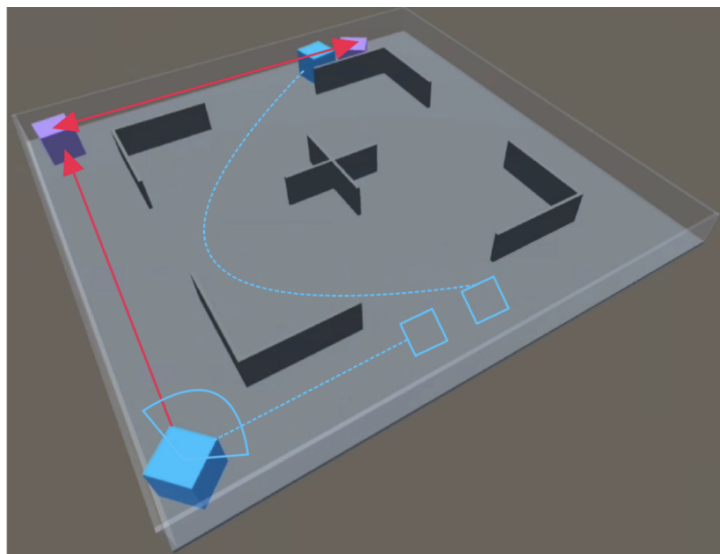


图 5.7 双人抓捕：策略仿真（一）



如上图所示，在游戏开始时抓捕方（蓝色）的一名智能体占住了外侧过道口，以此能看到两边的过道；另一名则直接移动到了逃脱方（紫色）的半区进行抓捕，最终形成二包一的态势。这说明了智能体已经初步形成了团队合作的习惯，如果只是单独的一个抓捕方去抓一个逃脱方，逃脱方可能会因为速度优势成功逃离；而抓捕方一旦有了团队协作的意识，则能够利用障碍物封锁逃脱方的所有路径，大大提高抓捕的成功率。

面对抓捕方的进攻，逃脱方在开局阶段会如下图所示那样来到中间进行躲避，一方面是因为中间区域的地形相较于四周的过道更加开阔，方便逃跑；另一方面是因为中间也更容易利用障碍物来遮蔽抓捕方的视线。这也说明了在训练过程中，双方智能体都会根据对方所采取的策略来进一步调整自身的行为选择，而非一成不变。

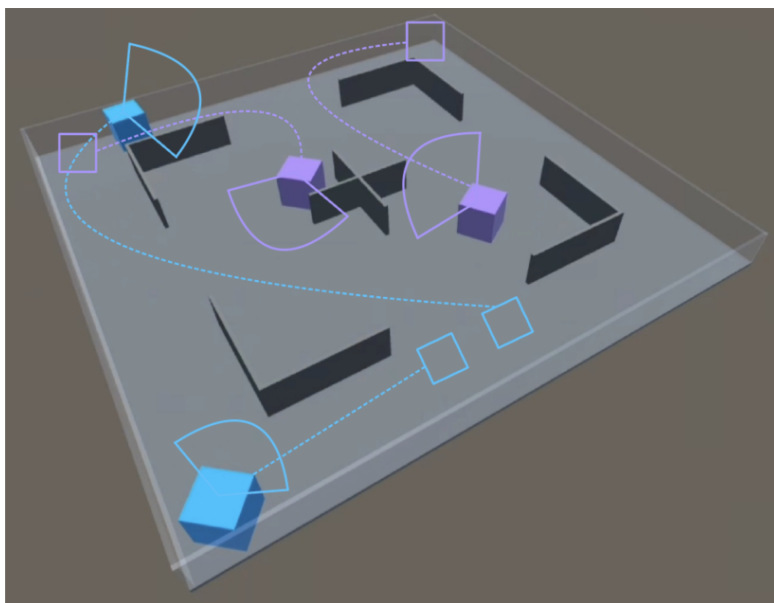


图 5.8 双人抓捕：策略仿真（二）

如下图所示，即使在一名逃脱方的智能体被抓捕后，另一名智能体依然会尽可能地躲避追捕。逃脱方在视野中发现追捕方后，会第一时间绕到障碍物后侧，避免被抓捕方所发现。另外，逃脱方在游戏过程偶尔会出现离开当前位置，并且去主动寻找抓捕方所在的位置。这种策略是在设计阶段没有预想到的，但确实是一种风险与收益并存的行为，一来可能会因此暴露位置给抓捕方，但也能帮助逃脱方及时掌握抓捕方的信息，从而调整自己的位置。

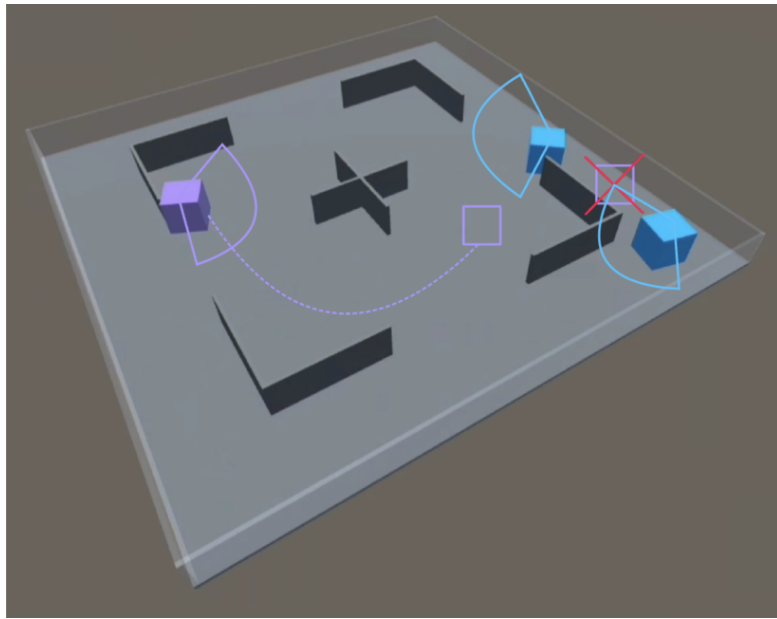


图 5.9 双人抓捕：策略仿真（三）

5.4 实验三：三人足球

在三人足球实验中，由于双方智能体的行为模式相同，因此可以直接根据 ELO 评分来衡量智能体的决策水平。如图 (5.10) 所示，智能体的 ELO 分数在实验过程中稳步增长，在六百万步时达到 1250，在八百万步时达到 1350 左右并逐渐收敛维持在了该水平区间。

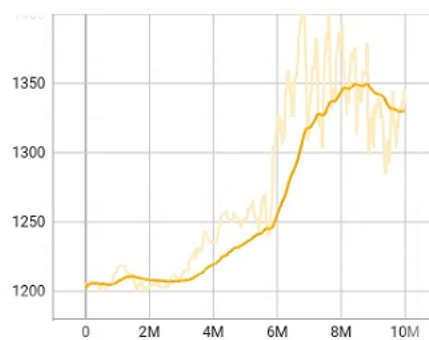


图 5.10 三人足球：ELO 评分

策略仿真方面，虽然在设计阶段没有赋予智能体前锋和守门员的职责区分，但是经过训练后智能体仍然理解位置的重要性：靠近己方球门的智能体扮演起了守门员的职能，仍然停留在球门附近作防守；而靠近中场的智能体扮演了前锋的角色发起进攻，同时也找到了对方的一名队员，实施盯人战术。

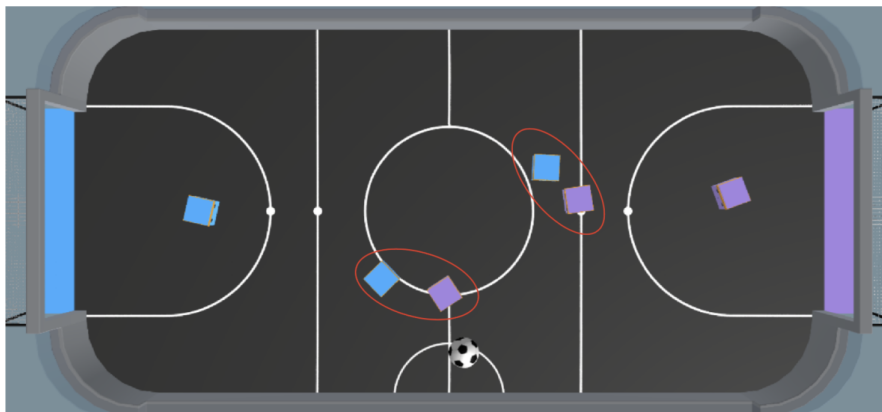


图 5.11 三人足球：策略仿真（一）

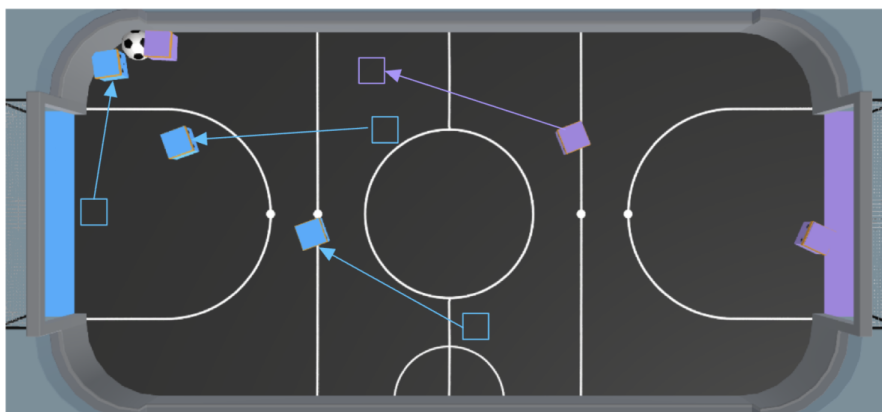


图 5.12 三人足球：策略仿真（二）

如图 (5.12) 所示，当有紫色方的一名智能体将球带到前场后，蓝色方靠近球门的智能体能够第一时间上前阻挡，其余队员则立刻回到己方半区一同防守；而紫色方的另一名智能体会适当前压，守门员则仍然待在原地防守。



第六章 结论与展望

6.1 主要结论

在多人对抗性游戏环境中创建并训练具备特定行为逻辑的智能体，对于当前电子游戏产业的发展具有显著的研究与应用价值，本文主要研究成果如下：

1、本文基于 Unity3D 游戏引擎以及 Unity-ML 工具包，搭建并部署了本地的强化学习环境，为后续游戏智能体的强化学习研究和实现奠定了基础。

2、本文设计并制定了从易到难的三种不同的对抗性实验场景。第一个场景为双人射击游戏，主要用来考察智能体的基础射击与躲避能力；第二个场景为双人抓捕游戏，着重考验智能体的策略规划与迅速反应能力；第三个场景则是经典的三人足球游戏，旨在测试智能体之间的团队协作和动作执行能力。对于每一个实验场景，都加入了独特的环境元素以增添游戏的复杂性，同时对不同智能体的状态空间、动作空间以及奖励函数做了合理的区分和设计。

3、本文选用了 PPO 算法，结合课程学习的策略，利用“中心化训练”+“去中心化决策”的架构，验证了深度强化学习在设计环境中的有效性。针对对抗性游戏中具有相同属性的智能体，通过引入 ELO 评分机制，侧面反映出智能体的相对竞争水平，还能在训练过程中存储不同决策能力的智能体模型。

4、本文利用训练完成后的模型在 Unity 平台上的进一步仿真检验，发现：在双人射击游戏中，智能体展示出了躲避子弹以及利用掩体的策略；在双人抓捕游戏中，抓捕方的智能体学会了策略分工与合作，逃脱方的智能体学会了利用障碍物躲避视野；而在三人足球游戏中，智能体会根据球的位置，及时进攻前压或后撤回防，证明了强化学习算法在对抗性游戏中指导多智能体掌握开发者所希望的合理策略的有效性，并且能够自动调整策略以应对不断变化的对手行为和游戏状态。

5、本文在策略仿真阶段，发现了智能体会训练出开发者意料之外的策略。比如：在双人射击游戏中，智能体会选择冲到前场发射子弹，减少对手的反应时间；在双人抓捕游戏中，逃脱方会选择离开暂时安全的地点，主动去寻找抓捕方的位置，便于自身后续的行为选择；在三人足球游戏中，智能体甚至能在没有明确分配特定角色的情况下，形成前锋、守门员的角色感知。这也侧面反映了强化学习在对抗性游戏领域具有极大的潜力和适应性，智能体通过不断试验和错误的方式，能够探索出非常规解决方案，甚至能够在非预设的游戏环境中找到优



势。



6.2 研究展望

虽然本文最终的实验取得了显著的效果，但是在实验过程中仍然发现了较多问题，智能体在训练到收敛一段时间后会发现 ELO 分值连续下降的情况、部分智能体的训练时间会达到一天甚至更长等。基于此，本课题的后续研究会主要包括：

- 1、合理利用多层传感器，设定更加符合场景的数据输入，使智能体能够通过更少数据达到理想的训练效果。
- 2、设定更加合理的奖励函数，使智能体的行为逻辑更加接近人类，并利用课程学习的阶梯式进步法，减少策略收敛所需要花费的时间。
- 3、由于本文主要研究的是强化学习，故使用的神经网络均是 MLP，后续可以优化神经网络模型，将多种网络进行融合，以提升训练效果。



参考文献

- [1] VINYALS O, BABUSCHKIN I, CZARNECKI W M, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning[J]. Nature, 2019, 575(7782): 350–354.
- [2] NING Z, XIE L. A survey on multi-agent reinforcement learning and its application[Z]. [S.l.: s.n.].
- [3] HERNANDEZ-LEAL P, KARTAL B, TAYLOR M E. Is multiagent deep reinforcement learning the answer or the question? A brief survey[J]. CoRR, 2018.
- [4] SUTTON R, MCALLESTER D, SINGH S, et al. Policy gradient methods for reinforcement learning with function approximation[Z]. [S.l.: s.n.], 1999.
- [5] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms [J]. CoRR, 2017.
- [6] KONDA V, TSITSIKLIS J. Actor-critic algorithms[C]//SOLLA S, LEEN T, MÜLLER K. Advances in Neural Information Processing Systems: volume 12. [S.l.]: MIT Press, 1999.
- [7] 王琦, 杨毅远, 江季. Easy rl: 强化学习教程[M]. 北京: 人民邮电出版社, 2022.
- [8] 俞勇教授团队. 动手学强化学习[M]. 北京: 人民邮电出版社, 2022.
- [9] TAMPUU A, MATIISEN T, KODELJA D, et al. Multiagent cooperation and competition with deep reinforcement learning[J]. CoRR, 2015.
- [10] 王树森, 张志华等. 深度强化学习[M]. 北京: 人民邮电出版社, 2022.
- [11] LOWE R, WU Y, TAMAR A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments[J]. CoRR, 2017.
- [12] JULIANI A, BERGES V P, TENG E, et al. Unity: A general platform for intelligent agents[J]. arXiv preprint arXiv:1809.02627, 2020.
- [13] VIGÁRIO B, MAGALHÃES A, FREITAS F. Modern computer games technology in systems and control education[C]//7th Portuguese Conference on Automatic Control. [S.l.]: Citeseer, 2006.
- [14] SILVER D, HUBERT T, SCHRITTWIESER J, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play[J]. Science, 2018, 362(6419): 1140–1144.
- [15] ALBERS P C, DE VRIES H. Elo-rating as a tool in the sequential estimation of dominance strengths[J]. Animal Behaviour, 2001, 61(2): 489–495.
- [16] BANSAL T, PACHOCKI J, SIDOR S, et al. Emergent complexity via multi-agent competition. [J]. arXiv: Artificial Intelligence, arXiv: Artificial Intelligence, 2017.
- [17] BAKER B, KANITSCHIEDER I, MARKOV T, et al. Emergent tool use from multi-agent autotricula.[J]. arXiv: Learning, arXiv: Learning, 2019.